

09 Prepare: Text Files

Most computers permanently store lots of data on devices such as hard drives, solid state drives, and thumb drives. The data that is stored on these devices is organized into files. Just as a human can write words on a paper, a computer can store words and other data in a file. During this lesson, you will learn how to write Python code that reads text from text files.

Concepts

Broadly speaking, there are two types of files: text files and binary files. A **text file** stores words and numbers as human readable text. A **binary file** stores pictures, diagrams, sounds, music, movies, and other media as numbers in a format that is not directly readable by humans.

Text Files

In order to read data from a text file, the file must exist on one of the computer's drives, and your program must do these three things:

1. Open the file for reading text
2. Read from the file, usually one line of text at a time
3. Close the file

The built-in `open` function opens a file for reading or writing. Here is an excerpt from the official documentation for the `open` function:

```
| open(filename, mode="rt")
```

Open a file and return a corresponding file object.

filename is the name of the file to be opened.

mode is an optional string that specifies the mode in which the file will be opened. It defaults to "rt" which means open for reading in text mode. Other common values are "wt" for writing a text file (truncating the file if it already exists), and "at" for appending to the end of a text file.

Example 1 contains a program that opens a text file named `plants.txt` for reading at line 26. At line 30, there is a `for` loop that reads the text in the file one line at a time and repeats the body of the `for` loop once for each line of text in the file. In the body of the `for` loop at lines 32-38, the code removes surrounding white space, if there is any, from each line of text and then stores each line of text in a list.

```
1 # Example 1
2
```

```

3 def main():
4     # Read the contents of a text file
5     # named plants.txt into a list.
6     text_list = read_list("plants.txt")
7
8     # Print the entire list.
9     print(text_list)
10
11
12 def read_list(filename):
13     """Read the contents of a text file into a list and
14     return the list. Each element in the list will contain
15     one line of text from the text file.
16
17     Parameter filename: the name of the text file to read
18     Return: a list of strings
19     """
20     # Create an empty list that will store
21     # the lines of text from the text file.
22     text_list = []
23
24     # Open the text file for reading and store a reference
25     # to the opened file in a variable named text_file.
26     with open(filename, "rt") as text_file:
27
28         # Read the contents of the text
29         # file one line at a time.
30         for line in text_file:
31
32             # Remove white space, if there is any,
33             # from the beginning and end of the line.
34             clean_line = line.strip()
35
36             # Append the clean line of text
37             # onto the end of the list.
38             text_list.append(clean_line)
39
40     # Return the list that contains the lines of text.
41     return text_list
42
43
44 # Call main to start this program.
45 if __name__ == "__main__":
46     main()

```

```

> python example_1.py
['baobab', 'kangaroo paw', 'eucalyptus', 'heliconia', 'tulip',
'chupasangre cactus', 'prickly pear cactus', 'ginkgo biloba']

```

After the body of a for loop that reads from a file, we can write a call to the `file.close` method. However, when calling the `open` function, most programmers use a `with` block as shown in example 1 at [line 26](#) and nest the for loop inside the `with` block as shown at [lines 30–38](#). When the `with` block ends, the computer will automatically close the file, so that the programmer doesn't have to write a call to the `file.close` method.

CSV Files

Many computer systems import and export data in CSV files. CSV is an acronym for comma separated values. A **CSV file** is a text file that contains tabular data with each row on a separate line

of the file and each cell (column) separated by a comma. The following example shows the contents of a CSV file named `hymns.csv` that stores data about religious songs. Notice that the first row of the file contains column headings, the next four rows contain data about four hymns, and each row contains three columns separated by commas.

```
Title,Author,Composer
O Holy Night,John Dwight,Adolphe Adam
Away in a Manger,Anonymous,William Kirkpatrick
Joy to the World,Isaac Watts,George Handel
With Wondering Awe,Anonymous,Anonymous
```

Python has a standard module named `csv` that includes functionality to read from and write to CSV files. The program in example 2 shows how to open a CSV file and use the `csv` module to read the data and print it to a terminal window. In example 2 at line 8, there is a call to the Python built-in `open` function, which opens the `hymns.csv` file for reading. At line 12, the program creates a `csv.reader` object that will read from the `hymns.csv` file. Within the `for` loop at lines 16 and 17 the `csv.reader` reads and prints each row from the CSV file.

```
1  # Example 2
2
3  import csv
4
5  def main():
6      # Open the CSV file for reading and store a reference
7      # to the opened file in a variable named csv_file.
8      with open("hymns.csv", "rt") as csv_file:
9
10         # Use the csv module to create a reader object
11         # that will read from the opened CSV file.
12         reader = csv.reader(csv_file)
13
14         # Read the rows in the CSV file one row at a time.
15         # The reader object returns each row as a list.
16         for row_list in reader:
17             print(row_list)
18
19
20 # Call main to start this program.
21 if __name__ == "__main__":
22     main()
```

```
> python example_2.py
['Title', 'Author', 'Composer']
['O Holy Night', 'John Dwight', 'Adolphe Adam']
['Away in a Manger', 'Anonymous', 'William Kirkpatrick']
['Joy to the World', 'Isaac Watts', 'George Handel']
['With Wondering Awe', 'Anonymous', 'Anonymous']
```

When a `csv.reader` reads a row from a CSV file, the reader returns the row as a list of strings. The output from example 2 shows that a `csv.reader` returns a list of strings. In the output, notice the five lists of strings, (strings surrounded by square brackets `[ ... ]`) that were printed by the `print` statement at line 17. Notice also that the reader reads all the rows from a CSV file, including the first row, which contains column headings.

You might recall that in CSE 110, you wrote a program that reads from a CSV file without using a `csv.reader`. That program splits each row of text from the CSV file using the string `split` method.

Unfortunately, using the `split` method will not work for all CSV files. Consider the following `hymns.csv` file that contains rows for the hymns "Far, Far Way on Judea's Plains" and "Oh, Come, All Ye Faithful". Both of these hymns have commas in their titles. If we use the string `split` method to separate the columns in this CSV file, the hymn titles will be split. A `csv.reader` will correctly split rows in all valid CSV files.

```
Title,Author,Composer
"Far, Far Way on Judea's Plains",John Mcfarlane,John Mcfarlane
"Oh, Come, All Ye Faithful",John Wade,John Wade
"Christ the Lord is Risen Today",Charles Wesley,Anonymous
```

Processing Each Row in a CSV File

After reading each row from a CSV file, the `for` loop in the previous example simply prints the row list to a terminal window. Of course, a `for` loop can do much more than simply print each row. Consider the following CSV file named `dentists.csv` that stores data about dental offices. Notice that the first row of the file contains column headings, the next four rows contain data about four dental offices, and each row contains five columns separated by commas.

```
Company Name,Address,Phone Number,Employees,Patients
Eagle Rock Dental Care,556 Trejo Suite C,208-359-2224,7,1205
Apple Tree Dental,33 Winn Drive Suite 2,208-359-1500,10,1520
Rockhouse Dentistry,106 E 1st N,208-356-5600,12,1982
Cornerstone Family Dental,44 S Center Street,208-356-4240,8,1453
```

The program in example 3 processes each row in the `dentists.csv` file to determine which dental office has the most patients per employee. Notice that the first row of the `dentists.csv` file contains column headings. The headings contain no numbers and aren't needed for the calculations, so the program skips the first row by calling the built-in `next` function at [line 25](#).

```
1  # Example 3
2
3  import csv
4
5  # Indexes of some of the columns
6  # in the dentists.csv file.
7  COMPANY_NAME_INDEX = 0
8  NUM_EMPS_INDEX = 3
9  NUM_PATIENTS_INDEX = 4
10
11
12 def main():
13     # Open a file named dentists.csv and store a reference
14     # to the opened file in a variable named dentists_file.
15     with open("dentists.csv", "rt") as dentists_file:
16
17         # Use the csv module to create a reader
18         # object that will read from the opened file.
19         reader = csv.reader(dentists_file)
20
21         # The first row of the CSV file contains column
22         # headings and not data about a dental office,
23         # so this statement skips the first row of the
24         # CSV file.
25         next(reader)
26
```

```

27     running_max = 0
28     most_office = None
29
30     # Read each row in the CSV file one at a time.
31     # The reader object returns each row as a list.
32     for row_list in reader:
33
34         # For the current row, retrieve the
35         # values in columns 0, 3, and 4.
36         company = row_list[COMPANY_NAME_INDEX]
37         num_employees = int(row_list[NUM_EMPS_INDEX])
38         num_patients = int(row_list[NUM_PATIENTS_INDEX])
39
40         # Compute the number of patients per
41         # employee for the current dental office.
42         patients_per_emp = num_patients / num_employees
43
44         # If the current dental office has more
45         # patients per employee than the running
46         # maximum, assign running_max and most_office
47         # to be the current dental office.
48         if patients_per_emp > running_max:
49             running_max = patients_per_emp
50             most_office = company
51
52     # Print the results for the user to see.
53     print(f"{most_office} has {running_max:.1f}"
54           " patients per employee")
55
56
57 # Call main to start this program.
58 if __name__ == "__main__":
59     main()

```

```

> python example_3.py
Cornerstone Family Dental has 181.6 patients per employee

```

Reading a CSV File into a Compound List

The program in example 3 reads and processes each row in a CSV file. That program needs to access the data in each row once only. If a program needs to access the contents of a CSV file multiple times, the program can read the contents of the file into a compound list and then access the data from the list. The program in example 4 contains a function named `read_compound_list` that reads the contents of a CSV file into a compound list.

```

1  # Example 4
2
3  import csv
4
5  def main():
6      # Read the contents of the dentists.csv file
7      # into a compound list.
8      dentists_list = read_compound_list("dentists.csv")
9
10     # Print the entire list.
11     print(dentists_list)
12
13
14 def read_compound_list(filename):

```

```

15     """Read the contents of a CSV file into a compound
16     list and return the list. Each element in the
17     compound list will be a small list that contains
18     the values from one row of the CSV file.
19
20     Parameter filename: the name of the CSV file to read
21     Return: a list of lists that contain strings
22     """
23     # Create an empty list that will
24     # store the data from the CSV file.
25     compound_list = []
26
27     # Open the CSV file for reading and store a reference
28     # to the opened file in a variable named csv_file.
29     with open(filename, "rt") as csv_file:
30
31         # Use the csv module to create a reader object
32         # that will read from the opened CSV file.
33         reader = csv.reader(csv_file)
34
35         # Read the rows in the CSV file one row at a time.
36         # The reader object returns each row as a list.
37         for row_list in reader:
38
39             # Append one row from the CSV file to the
40             # compound list.
41             compound_list.append(row_list)
42
43     # Return the compound list.
44     return compound_list
45
46
47 # Call main to start this program.
48 if __name__ == "__main__":
49     main()

```

```

> python example_4.py
[['Company Name', 'Address', 'Phone Number', 'Employees',
'Patients'], ['Eagle Rock Dental Care', '556 Trejo Suite C',
'208-359-2224', '7', '1205'], ['Apple Tree Dental',
'33 Winn Drive Suite 2', '208-359-1500', '10', '1520'],
['Rockhouse Dentistry', '106 E 1st N', '208-356-5600', '12',
'1982'], ['Cornerstone Family Dental', '44 S Center Street',
'208-356-4240', '8', '1453']]

```

Reading a CSV File into a Compound Dictionary

If the values in one of the columns of a CSV file are unique, then a program can read the contents of a CSV file into a compound dictionary and then use the dictionary to quickly find data. Recall that each item in a dictionary is a key value pair. The values from the unique column in a CSV file will be the keys in the dictionary. The program in example 5 shows how to read the data from a CSV file into a compound dictionary. Notice in example 5, because of lines 9, 14, 54, and 58, that the program uses the dental office phone numbers as the keys in the dictionary.

```

1  # Example 5
2
3  import csv
4
5

```

```

6 def main():
7     # Index of the phone number column
8     # in the dentists.csv file.
9     PHONE_INDEX = 2
10
11     # Read the contents of the dentists.csv into a
12     # compound dictionary named dentists_dict. Use
13     # the phone numbers as the keys in the dictionary.
14     dentists_dict = read_dict("dentists.csv", PHONE_INDEX)
15
16     # Print the dentists compound dictionary.
17     print(dentists_dict)
18
19
20 def read_dict(filename, key_column_index):
21     """Read the contents of a CSV file into a compound
22     dictionary and return the dictionary.
23
24     Parameters
25         filename: the name of the CSV file to read.
26         key_column_index: the index of the column
27             to use as the keys in the dictionary.
28     Return: a compound dictionary that contains
29         the contents of the CSV file.
30     """
31     # Create an empty dictionary that will
32     # store the data from the CSV file.
33     dictionary = {}
34
35     # Open the CSV file for reading and store a reference
36     # to the opened file in a variable named csv_file.
37     with open(filename, "rt") as csv_file:
38
39         # Use the csv module to create a reader object
40         # that will read from the opened CSV file.
41         reader = csv.reader(csv_file)
42
43         # The first row of the CSV file contains column
44         # headings and not data, so this statement skips
45         # the first row of the CSV file.
46         next(reader)
47
48         # Read the rows in the CSV file one row at a time.
49         # The reader object returns each row as a list.
50         for row_list in reader:
51
52             # From the current row, retrieve the data
53             # from the column that contains the key.
54             key = row_list[key_column_index]
55
56             # Store the data from the current row
57             # into the dictionary.
58             dictionary[key] = row_list
59
60     # Return the dictionary.
61     return dictionary
62
63
64 # Call main to start this program.
65 if __name__ == "__main__":
66     main()

```

```
> python example_5.py
{'208-359-2224': ['Eagle Rock Dental Care', '556 Trejo Suite...'],
'208-359-1500': ['Apple Tree Dental', '33 Winn Drive Suite 2...'],
'208-356-5600': ['Rockhouse Dentistry', '106 E 1st N', '208-...'],
'208-356-4240': ['Cornerstone Family Dental', '44 S Center S...']}
```

Tutorials

If any of the concepts or topics in the previous section seem unfamiliar to you, reading these tutorials may help you understand the concepts better.

[Python "for" Loops](#)

[Reading and Writing CSV Files in Python](#)

Summary

A text file stores words and numbers as human readable text. During this lesson, you are learning how to write Python code to read from text files. To read from a text file, your program must first open the file by calling the built-in `open` function. You should write the code to open a file in a Python `with` block because the computer will automatically close the file when the `with` block ends, and you won't need to remember to write code to close the file.

A CSV file is a text file that contains rows and columns of data. CSV is an acronym that stands for comma separated values. Within each row in a CSV file, the data values are separated by commas. Python includes a standard module named `csv` that helps us easily write code to read from CSV files. Sometimes a program simply needs to use the values in a CSV file in calculations, so we write Python code to perform calculations for each row. Other times, we write Python code to read the contents of a CSV file into a compound list or compound dictionary.

Copyright © 2020, Brigham Young University - Idaho. All rights reserved.